

EXPERIMENT 4:

Aim

To write a Python program that solves Cryptarithmic puzzles (e.g., SEND + MORE = MONEY) using Backtracking and Constraint Satisfaction Problem (CSP) concepts.

Objective

To map each unique letter in a cryptarithmic equation to a unique digit from 0 to 9.

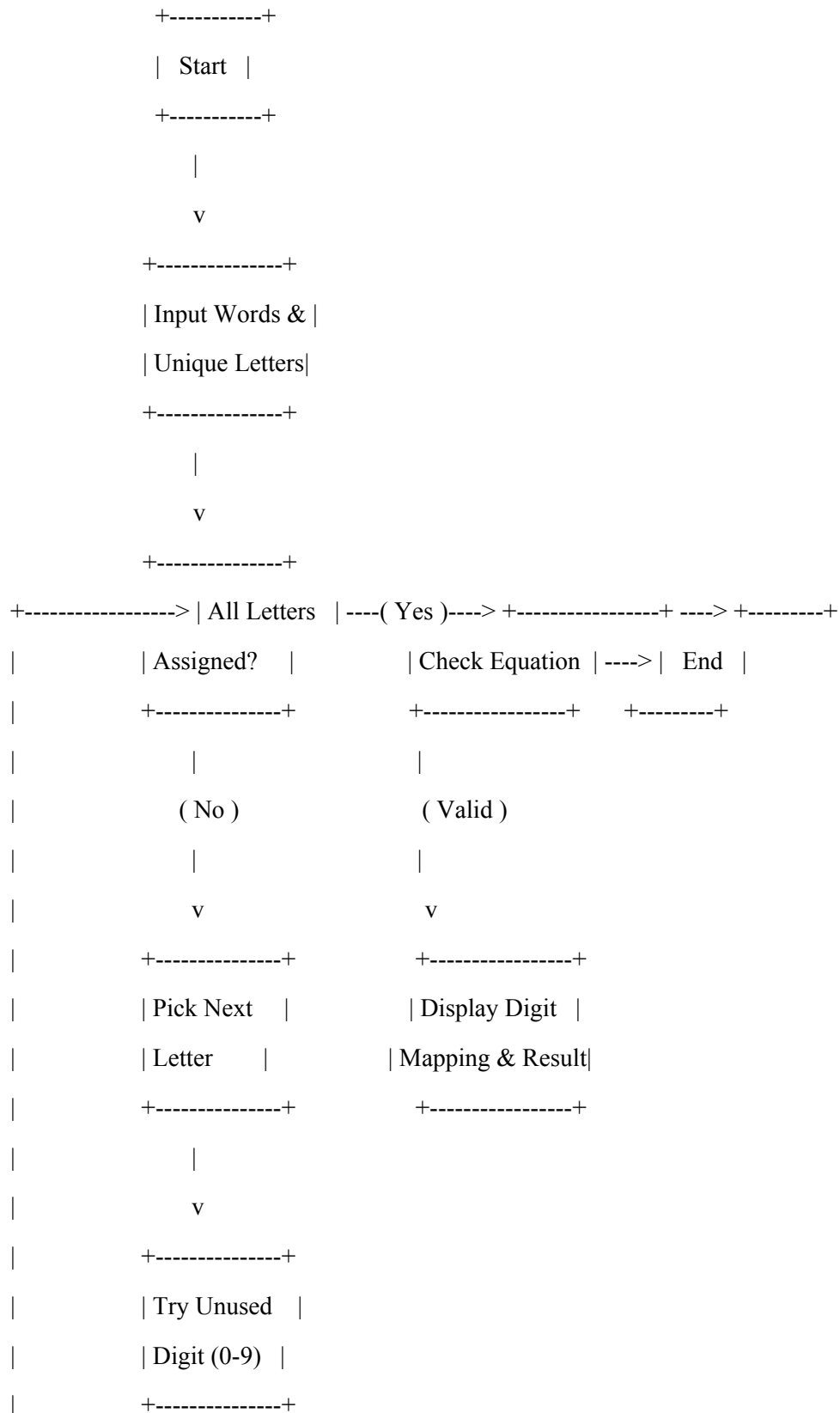
To enforce constraints ensuring no two letters share the same digit and leading letters are non-zero.

To find and display the valid digit substitution that satisfies the arithmetic equation.

Algorithm

1. Extract all unique letters from the given words in the equation.
2. If the total number of unique letters exceeds 10, return no solution (since digits are only 0 to 9).
3. Identify leading letters of each word and set a constraint that they cannot be assigned 0.
4. Use a backtracking algorithm to recursively map unassigned letters to available digits:
5. Select an unassigned letter.
6. Try assigning an unused digit (0 to 9) to the letter, checking non-zero leading constraints.
7. Recur to assign digits for the remaining letters.
8. When all letters are assigned digits, convert words into their numeric values.
9. Verify if the equation holds true (e.g., $\text{Value}(\text{Word1}) + \text{Value}(\text{Word2}) = \text{Value}(\text{Word3})$).
10. If valid, print the digit assignments and arithmetic solution; otherwise, backtrack and try next digit.

Flowchart



```

|           |
|           v
|   +-----+
|   | Check Leading |
|   | Non-Zero Rule |
|   +-----+
|       /      \
|   ( Safe )  ( Unsafe )
|       /      \
|       v       v
|   +-----+ +-----+
|   | Assign Digit | | Try Next Digit / |
|   | & Recur     | | Backtrack     |
|   +-----+ +-----+
|       |           |
|       v           |
+-----+| Next Letter |<-----+

```

Code

```
from itertools import permutations

def solve_cryptarithmic(w1, w2, w3):
    unique_letters = list(set(w1 + w2 + w3))
    if len(unique_letters) > 10:
        return None

    first_letters = {w1[0], w2[0], w3[0]}

    for perm in permutations(range(10), len(unique_letters)):
        mapping = dict(zip(unique_letters, perm))

        if any(mapping[char] == 0 for char in first_letters):
            continue

        num1 = int("".join(str(mapping[c]) for c in w1))
        num2 = int("".join(str(mapping[c]) for c in w2))
        num3 = int("".join(str(mapping[c]) for c in w3))

        if num1 + num2 == num3:
            return mapping, num1, num2, num3

    return None

if name == "main":
    w1, w2, w3 = "SEND", "MORE", "MONEY"
    print(f"Solving Cryptarithmic Problem: {w1} + {w2} = {w3}\n")
    result = solve_cryptarithmic(w1, w2, w3)

    if result:
        mapping, n1, n2, n3 = result
```

```

print("Letter Mapping:")
for char, digit in sorted(mapping.items()):
    print(f"{char} = {digit}")

print(f"\nEquation Solution:\n {n1}\n+ {n2}\n-----\n {n3}")
else:
    print("No solution exists.")

```

Output

Solving Cryptarithmic Problem: SEND + MORE = MONEY

Letter Mapping:

D = 7

E = 5

M = 1

N = 6

O = 0

R = 8

S = 9

Y = 2

Equation Solution:

9567

• 1085

10652

Result

The Cryptarithmic program was executed successfully. By evaluating unique letter combinations and enforcing positional arithmetic constraints, it accurately mapped each letter to a unique digit satisfying $SEND + MORE = MONEY$.

Conclusion

Cryptarithmic problems are effectively modeled as Constraint Satisfaction Problems (CSPs). Using backtracking with permutation mapping allows systematic elimination of invalid digit assignments, providing an exact mathematical solution.